

Streaming Latency Benchmarks for Real-Time Football Feeds: REDIS Streams versus Apache KAFKA, and the Consistency Check That Overturned Our First Answer

Journal Title
XX(X):1–17
©The Author(s) 2026
Reprints and permission:
sagepub.co.uk/journalsPermissions.nav
DOI: 10.1177/ToBeAssigned
www.sagepub.com/

SAGE

Gustavo Pedro Ricou¹

Abstract

Aim. Live football analytics depends on streaming middleware, and teams choose between Apache KAFKA and REDIS Streams using benchmarks run at data rates no sport produces. We set out to compare the two on real football feeds, under the number of simultaneous matches football actually generates.

What we did. We replayed 3,315 matches from the StatsBomb open dataset (2003–2023) at their true event rate through both systems, on a four-machine cloud testbed, with open load-generation code. We measured 2,266 runs in total.

The unexpected result. Our first complete answer was physically impossible. It showed REDIS slowing down as matches were added while KAFKA stayed flat, with very strong statistical support. Checking every run against a simple physical rule — a message cannot arrive before it is sent — showed that 58% of our runs contained impossible timestamps, including every run behind that finding. The faulty runs looked completely normal in every summary statistic we or a reviewer would inspect.

The expected results. After discarding the faulty runs, the two brokers are equivalent to within one millisecond and neither slows down as matches are added, which is what the sport's low data rate predicts. Two practical differences remain: end-to-end delay is 20 times lower for REDIS (5.2 vs 105.5 ms), caused by the client rather than the broker; and this reverses when the consumer sits across a network, where KAFKA stays under 100 ms and REDIS reaches 31 seconds unless one line of consumer code is changed.

Keywords

Real-time sports analytics, football, streaming systems, Apache Kafka, Redis Streams, latency benchmarking, measurement validity, reproducible research

1 Introduction

Live football analytics products — in-play odds engines, coaching dashboards, broadcast statistics overlays — all share the same shape. Events are recorded as the match happens, sent across a network, and delivered to a model that must react before the moment passes. The software that carries those events is called a message broker, and two products dominate the role.

Apache KAFKA [Kreps et al. \(2011\)](#) stores messages in a partitioned, replicated log written to disk. It is designed for durability and high throughput. REDIS Streams [Sanfilippo \(2017\)](#) keeps messages in memory in an append-only structure. It is designed for minimum delay on each operation. Practitioners generally choose between them using published comparisons that report REDIS as two to five times faster [Diaries \(2025\)](#); [Yerrapragada \(2025\)](#); [JusDB \(2025\)](#). None of those comparisons uses a sports workload, and most cannot be reproduced.

This paper asks the practical question directly. Using the StatsBomb open dataset [StatsBomb \(2023\)](#) and open load-generation scripts, we compare end-to-end delay for the two systems at the number of simultaneous matches football

actually produces. We then ask three follow-up questions that a practitioner needs answered: where in the pipeline does any difference come from, under which deployment conditions does the answer change, and how much configuration effort does each system demand.

1.1 The answer we obtained first

Our first campaign returned a clean result. REDIS's broker delay rose steadily as we added concurrent matches (1.006 → 1.197 → 1.346 ms across one, five and ten feeds), while KAFKA stayed flat to within 0.3%. Every comparison was statistically significant after correction for multiple testing, the strongest at $p = 9.0 \times 10^{-11}$, and at five feeds and above the two systems' results did not overlap at all.

¹School of Computer Science and Statistics, Trinity College Dublin, Dublin, Ireland

Corresponding author:

Gustavo Pedro Ricou, School of Computer Science and Statistics, Trinity College Dublin, Dublin, Ireland.

Email: pedrorig@tcd.ie

The finding was not only significant, it was explicable. A REDIS server executes commands on a single thread, so concurrent streams must queue for one execution context, whereas KAFKA divides a topic into partitions that can be served in parallel. Our measurement appeared to reproduce the textbook difference between the two designs, in the expected direction and at the expected scale.

It was an artefact. We compute broker delay by subtracting the time the broker confirmed a message from the time the consumer received it. Those two times are recorded by two different programs. A message cannot arrive before it is sent, so a negative value is not measurement noise; it is proof that the instrument has failed. We checked every run in the study against that rule — not only the runs whose results looked odd — and found impossible timestamps in 862 of 1,382 single-machine runs and 459 of 884 multi-machine runs. That included every run behind the result above.

1.2 Why we make this the centre of the paper

Timestamp inversion of this kind is invisible to every check a reader would apply. The medians stay positive and plausible. The confidence intervals stay narrow. The effect sizes are large and the direction matches theory. Only 5 to 14% of individual events invert, which is enough to shift a median by a fraction of a millisecond in a measurement whose entire effect is a fraction of a millisecond, and far too little to make any summary look wrong.

A benchmark can therefore produce a complete, internally consistent and entirely false set of results, and no amount of statistical care will expose it. Only a check against a physical constraint will, and it has to be applied to every condition as a rule rather than to the conditions that look suspicious.

1.3 Contributions

1. **A clock-integrity check for streaming benchmarks**, stated as a rule rather than a judgement, applied to all 2,266 runs in this study and released as open code. We report what it rejects, why the rejected data looked correct, and one property that makes such benchmarks fragile in general: the check bites hardest exactly where the measured effect is smallest (Section 6).
2. **The benchmark itself, inside the check**. At football's real concurrency levels, REDIS Streams delivers a median end-to-end delay of 5.2 ms against KAFKA's 105.5 ms, while the two brokers themselves are equivalent to within one millisecond and neither slows down as matches are added (Section 7.1).
3. **An explanation and a configuration account**. The twentyfold difference comes from the client library, not the broker, and it only appears at football's low event rate. Each system has exactly one client setting worth one to two orders of magnitude in delay, and in each case the introductory example in the documentation uses the slow value (Sections 7.2 and 8.2).
4. **A description of the football event feed as a workload**, covering 3,315 matches, which supplies the event rate, burst structure and concurrency levels the benchmark is driven at (Section 3).

1.4 Research questions

- RQ1 (delay):** Replaying real match feeds at their true event rate, what end-to-end delay do KAFKA and REDIS Streams deliver, and do they differ?
- RQ2 (concurrency):** Does that delay change across the number of simultaneous matches football produces, and does either system degrade faster than the other?
- RQ3 (cause):** Where does any difference come from — the broker itself, or the client software on either side of it?
- RQ4 (deployment):** Does the answer survive when the consumer sits across a network, and what explains the failure if it does not?
- RQ5 (validity):** How much of a benchmark conducted in the usual way survives an explicit physical-consistency check, and what does the rejected portion look like beforehand?

We answer RQ5 first, in Sections 5 and 6, because its answer decides which evidence RQ1–RQ4 are allowed to use.

1.5 How the paper is organised

Section 2 reviews the four literatures this work draws on and states what each supplies. Section 3 describes the football event feed as a workload and derives the experiment's parameters from it. Section 4 describes the testbeds, the measurements and the integrity check. Section 5 presents the first result set in full, Section 6 audits it, and Section 7 reports what survives. Section 8 gives practical guidance and states what remains unresolved.

2 Background and Related Work

This study sits between four literatures. We set out what each contributes, and what it does not supply, so that every assumption made later has a stated source.

2.1 The two systems, and how their clients work

Stonebraker et al. [Stonebraker et al. \(2005\)](#) set out eight requirements for real-time stream processing, including keeping data moving and processing it with predictably low delay. These remain the standard against which streaming middleware is judged.

KAFKA [Kreps et al. \(2011\)](#) implements them as a distributed commit log. A topic is divided into partitions; each partition is an ordered, append-only sequence written to disk and replicated. Throughput comes from batching writes and reading sequentially; parallelism comes from assigning partitions to independent consumers. REDIS Streams [Sanfilippo \(2017\)](#) takes the opposite position: an in-memory append-only structure with consumer groups, trading storage capacity for the lowest possible delay per operation.

One architectural consequence matters throughout this paper. A REDIS server executes commands on a single thread, so concurrent streams contend for one execution context, whereas KAFKA's partitioning permits genuine

parallelism inside a broker. This is the prediction our invalid first result appeared to confirm (Section 5), and it is why that result was persuasive.

A second consequence concerns the client libraries rather than the servers, and it turns out to matter more. Both systems place substantial machinery on the client side. A KAFKA producer does not send each record immediately; it appends the record to an in-memory batch which a background sender thread transmits, governed by settings including `linger.ms` (how long to wait for a batch to fill) and `max.in.flight.requests.per.connection` (how many unacknowledged requests may be outstanding). A KAFKA consumer likewise serves `poll()` from a locally prefetched buffer and commits its position on a timer. A REDIS Streams consumer by contrast issues an explicit blocking read (`XREADGROUP`) and an explicit acknowledgement (`XACK`) per message, each of which is a separate request and reply. Vendor documentation for both systems discusses these settings [Confluent \(2025\)](#); [Labs \(2025\)](#). Our results in Sections 7.2 and 7.3 turn on them.

The CAP theorem [Brewer \(2012\)](#) and its extension PACELC [Abadi et al. \(2012\)](#) frame the durability–latency trade-off both systems expose. PACELC’s “else” clause — that even without a network partition a system trades latency against consistency — is what our durability comparison probes.

2.2 How streaming systems are benchmarked

Stream processing has a mature benchmark literature. Linear Road [Arasu et al. \(2004\)](#) is the canonical application benchmark, posing tolling and traffic queries over a simulated stream of vehicles. NEXMark [Tucker et al. \(2008\)](#) models an online auction and has become the standard query suite for streaming dataflow engines. ESPBench [Hesse et al. \(2021\)](#) constructs an enterprise workload spanning transactional and analytical operators. RIoTbench [Shukla et al. \(2017\)](#) supplies Internet-of-Things dataflows with realistic sensor arrival patterns. The OpenMessaging Benchmark [OpenMessaging Project, Linux Foundation \(2018\)](#) is the closest analogue to this work, measuring broker throughput and delay directly across messaging systems.

Three observations follow, and each shapes a design decision here.

First, every one of these benchmarks drives its system with a *synthetic* arrival process — generated vehicles, auctions, sensor readings, or fixed-rate publishers — chosen for analytical tractability rather than fidelity to a domain. That is the right choice for a general-purpose benchmark and the wrong one here. Our central explanatory result (Section 7.2) is that the difference between the two systems appears at football’s low event rate and *disappears* when the same data is replayed ten times faster. A fixed-rate synthetic publisher would have reported no difference at all.

Second, these benchmarks are built to locate the point at which a distributed system breaks. Football does not come close to one, as Section 3 shows, so the interesting question for this domain is not where the system breaks but whether it is anywhere near breaking.

Third, and most relevant to this paper, a recent survey of benchmark practice in KAFKA event-streaming systems [Mohammad \(2025\)](#) observes that published results

are frequently not comparable, because client configuration, acknowledgement semantics and measurement instrumentation differ between the systems under test. Our experience is a strong instance of that critique and extends it in one direction: we find that comparable *configuration* is not sufficient, because the measurement instrument itself can fail silently under load. We are not aware of a streaming benchmark that reports what fraction of its runs it discarded on integrity grounds. Given our findings, we take that to be a gap in reporting practice rather than evidence that the problem is rare.

2.3 Measuring time across processes

The failure at the centre of this paper is a timing failure, and the distributed systems literature supplies the vocabulary for it.

Lamport [Lamport \(1978\)](#) established that in a distributed system there is no single physical time, only a partial order induced by causality: if event *a* can influence event *b* then *a* must precede *b* in any correct account. Our check is the simplest possible application of that principle. A broker’s acknowledgement of a message causally precedes the consumer’s receipt of it, so a measurement in which receipt is timestamped earlier is not a small error but a violation of the causal order, and any conclusion drawn from it is unsupported.

In practice, systems approximate physical time using clock synchronisation protocols, of which the Network Time Protocol [Mills \(1991\)](#) is the most widely deployed. Spanner [Corbett et al. \(2013\)](#) takes the further step of exposing clock *uncertainty* to the application through an explicit interval API, on the grounds that a system which pretends to know the time exactly will eventually be wrong in a way it cannot detect. Our situation is milder — both of our processes run on one machine and read the same operating-system clock, so there is no synchronisation error to speak of — and this is precisely why the failure is instructive. The inversion arises not from clock skew but from *when the timestamp is taken*: a thread that is descheduled between the event it is recording and the call that records it will stamp a time later than the event. Under CPU saturation this delay is routinely larger than the interval being measured.

Jain [Jain \(1991\)](#) sets out the standard methodology for computer systems performance measurement, including the requirement that an instrument be validated on the system it will measure. The check we describe in Section 4.4 is best understood as a specific, cheap instrument validation for cross-process delay measurement.

2.4 Football event data as a workload

Live football event data is produced by trained human operators who watch the match and record each action through specialised software. Pappalardo et al. [Pappalardo et al. \(2019\)](#), in releasing a large public event dataset, describe this collection process directly. Liu et al. [Liu et al. \(2013\)](#) measured its reliability: two independent groups of experienced operators coding the same Spanish league match reached very good agreement on team events (Cohen’s κ of 0.92 and 0.94), with a mean difference in recorded event time of 0.06 ± 0.04 s.

Two things follow for this study. The reliability result establishes that event data is accurate enough for the timing comparisons we make, since operator disagreement about *when* an event occurred is around 60 ms and our replay uses the recorded times identically for both systems. But no peer-reviewed measurement exists of how long the annotation pipeline takes end to end — that is, the delay between an action on the pitch and the corresponding event becoming available to a consumer. Vendor descriptions of these pipelines report figures in seconds. Because those are claims rather than measurements, we do not adopt a point estimate; we sweep the value across a range and report what share of the budget each component owns throughout (Section 7.6).

The StatsBomb open dataset [StatsBomb \(2023\)](#) used here is a sample of matches released for research, covering multiple competitions and seasons. We treat it as a sample throughout, which constrains the concurrency claim in Section 3.2.

2.5 Latency requirements in sport, and where they come from

Practitioner sources state latency budgets for live sports applications. Betting operators require updates within a few hundred milliseconds for in-play odds [Solutions \(2025\)](#); [Ververica \(2025\)](#); broadcast overlays tolerate up to a few seconds [OptiView \(2025\)](#); [Promwad \(2025\)](#). Table 1 collects these.

We must be explicit about the status of that table, because a reader could reasonably mistake it for measured evidence. Every figure in it is *asserted* by a vendor or industry source, not derived from any published measurement of what missing the budget costs. We use it as the practitioner context our results are interpreted against, and not as a criterion of success. One consequence is worth stating at the outset: every configuration we measure soundly meets every budget in the table, and the interesting differences between the two systems are two orders of magnitude below the tightest of them — until a network hop is introduced, at which point one configuration misses every budget by four orders of magnitude.

We also draw on the Age-of-Information framing [Kaul et al. \(2012\)](#), which formalises the cost of acting on stale data: a monitor acting on information of age L incurs an error that grows with L . We use this only to express our measured milliseconds on a decision-relevant scale in Section 7.6. For that expression we need an in-play win-probability model. The state of the art for soccer is the Bayesian model of Robberechts et al. [Robberechts et al. \(2021\)](#), whose implementation is not public; we therefore use a transparent calibrated stand-in and are explicit about its limitations.

2.6 Statistical methods

Latency distributions are strongly non-normal, so we use rank-based tests throughout: the Mann–Whitney U test [Mann and Whitney \(1947\)](#) for two-group comparisons and the Kruskal–Wallis test [Kruskal and Wallis \(1952\)](#) across concurrency levels, with Holm’s step-down procedure [Holm \(1979\)](#) controlling the family-wise error rate.

Several of our claims are negative — that two systems do *not* differ. Failing to reject a null hypothesis is not evidence for it, so we state those claims using the two one-sided tests procedure [Schuirmann \(1987\)](#), which reverses the burden of proof by testing whether the difference is smaller than a pre-specified margin. Lakens [Lakens \(2017\)](#) gives an accessible treatment and the practical guidance we follow on choosing margins. As a point estimate of the difference between two systems we use the Hodges–Lehmann shift [Hodges and Lehmann \(1963\)](#), the median of all pairwise differences, with confidence intervals from the percentile bootstrap [Efron \(1979\)](#). Section 4.5 explains why we moved to this estimator from a difference of means.

2.7 What is missing, and what this paper adds

Streaming benchmarks measure delay but use synthetic workloads and do not report integrity audits. Sports systems literature asserts latency budgets without measuring the transport term inside them. The football event-data literature characterises annotation accuracy but not pipeline delay. This paper measures the transport term on a real football workload, reports the integrity audit that most of our own measurements failed, and states what a practitioner should do as a result.

3 The Football Event Feed as a Workload

A latency benchmark needs three parameters: how fast events arrive, how bursty they are, and how many feeds run at once. These are usually chosen for convenience. Because our main explanatory result depends on the arrival rate being football’s rather than a generator’s, we derive all three from the data first.

We use the StatsBomb open dataset across 2003–2023: 52 competition-seasons, 15 competitions and 3,315 matches, of which 2,806 are men’s and 509 women’s. The corpus is fetched from a pinned upstream commit by `scripts/fetch_statsbomb_corpus.py`, so it can be reconstructed exactly.

3.1 Arrival rate and burstiness

Football event data is sparse. A match produces a median of 3,507 events over roughly 8,412 seconds of match clock, a mean rate of **0.415 events per second** (Figure 1a). That is the number a capacity calculation would use, and on its own it is misleading. Measured over a sliding ten-second window, the same feeds peak at **2.70 events per second**. The ratio of peak to mean is **6.45** (Figure 1b, Table 2). Half of all gaps between events are one second or shorter, and the fifth percentile is indistinguishable from zero. Events cluster because football clusters: a restart, a sequence of quick passes, a goal and the play around it all arrive in tight groups separated by long quiet periods.

The pattern is stable across the corpus. Every competition falls between 0.35 and 0.44 events per second with a peak-to-mean ratio between 6.2 and 7.9, and the men’s and women’s game differ by less than seven percent on both measures. The sparsest competitions are the busiest: the African Cup of Nations and the Indian Super League have the lowest mean rates and the highest ratios, so a low average is not a licence to provision for the average.

Table 1. Latency budgets for live sports applications as asserted in vendor and practitioner sources. These are stated targets, not measured costs of missing them, and we use them as context rather than as a success criterion. Sources: [Solutions \(2025\)](#); [Ververica \(2025\)](#); [OptiView \(2025\)](#); [Promwad \(2025\)](#).

Use Case	Stakeholder	Stated Range	Stated Threshold
Betting Platforms			
Live Odds Update	Betting Platform	100–500ms	< 500ms
In-Play Betting	Betting Platform	50–200ms	< 200ms
Broadcasting			
Live Video Sync	Broadcaster	500–3000ms	< 3000ms
Live Stats Overlay	Broadcaster	100–1000ms	< 1000ms
Interactive Features	Broadcaster	200–500ms	< 500ms
Fan Applications			
Push Notifications	Fan	1000–3000ms	< 3000ms
Live Updates	Fan	500–2000ms	< 2000ms

Two consequences carry into the rest of the paper. A buffer sized from the mean is undersized by more than a factor of six. More importantly, a feed that is idle most of the time and busy in short bursts is exactly the pattern under which a client that must wake from idle pays that cost on every burst. Section 7.2 measures a 103 ms cost of precisely that shape.

3.2 How many matches run at once

Concurrency levels in this literature are chosen by hand. Our own earlier design swept one, five, ten and twenty feeds, and later ten, twenty-five, fifty and one hundred. That makes “does delay scale with concurrency?” a question about an invented variable. Football supplies the variable directly, because match records carry a kick-off date and time.

Across 2,346 distinct kick-off instants in the corpus, the largest carries **12 simultaneous matches**, and allowing for a nominal 115-minute match the peak number of matches *in play* at any instant is **21** (Figure 2). The median slot carries one match. These maxima are not accidents of sampling: domestic leagues schedule the final matchday simultaneously to protect competitive integrity, so a 20-team league plays ten concurrent matches by rule, and our largest observed slots are exactly such fixtures.

We therefore benchmark at $N \in \{1, 9, 10, 12\}$: the median slot, the upper quartile of occupancy, the structural league-matchday bound, and the largest simultaneity observed.

Two limits on this must be restated wherever the numbers are used. First, StatsBomb open data is a *sample* of matches, not a complete record of any league-season, so observed simultaneity is a **lower bound**; the structural bound of half the teams in a league is the safer planning figure. Second, that bound is *per league*. A commercial feed platform serving many leagues and divisions at once faces their sum, which on a European Saturday afternoon runs into the hundreds of matches. Our concurrency result therefore describes a consumer of a single competition. The connection experiment of Section 7.4 is the arm that would speak to the multi-league case, and it is the arm the integrity check largely removed.

4 Method

4.1 Two testbeds, both disclosed

Several numbers in earlier versions of this work turned out to be properties of the machine rather than of either broker, so we state both testbeds fully and measure the parts that limit resolution (`scripts/platform_probe.py`; raw output in `docs/results/platform/`).

Testbed A (single machine). Windows 11 (build 10.0.26200), 8-core AMD Ryzen, 31 GB RAM, CPython 3.9.13. The producer and consumer are native Windows processes; KAFKA 4.1.1 and REDIS 7.2.4 run in Docker containers whose engine executes inside a WSL2 virtual machine, reached through published ports. Two measured floors bound everything measured here. A requested 1 ms sleep takes a median of **15.6 ms**, because of the Windows timer tick, so event emission is quantised to a roughly 15.6 ms grid. And opening a TCP connection to a published container port costs a median of 5.6 ms for REDIS and 9.9 ms for KAFKA, far above a genuine loopback path.

Testbed B (four machines). Four Oracle Cloud VM.Standard.E5.Flex virtual machines in the London region — three brokers and one driver — running Ubuntu 22.04 and CPython 3.10.12, communicating over the tenancy’s private network. Client library versions are pinned identically to Testbed A. Both floors disappear: a requested 1 ms sleep takes 1.06 ms, and a round trip to the broker on an established connection is 0.22 ms on the same host and 0.24 ms across machines. A genuine two-machine network is therefore *faster* than Testbed A’s “co-located” path, which is the sharpest statement we can offer of how much a testbed can distort a benchmark.

Every result reported as a finding in Section 7 comes from Testbed B. Testbed A appears in Section 5 because it produced the result set this paper is about.

4.2 Replay and load generation

Each match is preprocessed into a CSV *replay plan* that records, for every event, the time at which it should be emitted (`scripts/make_replay_plan.py`). A producer replays a plan at a configurable speed-up factor; a consumer reads the events and records when they arrive.

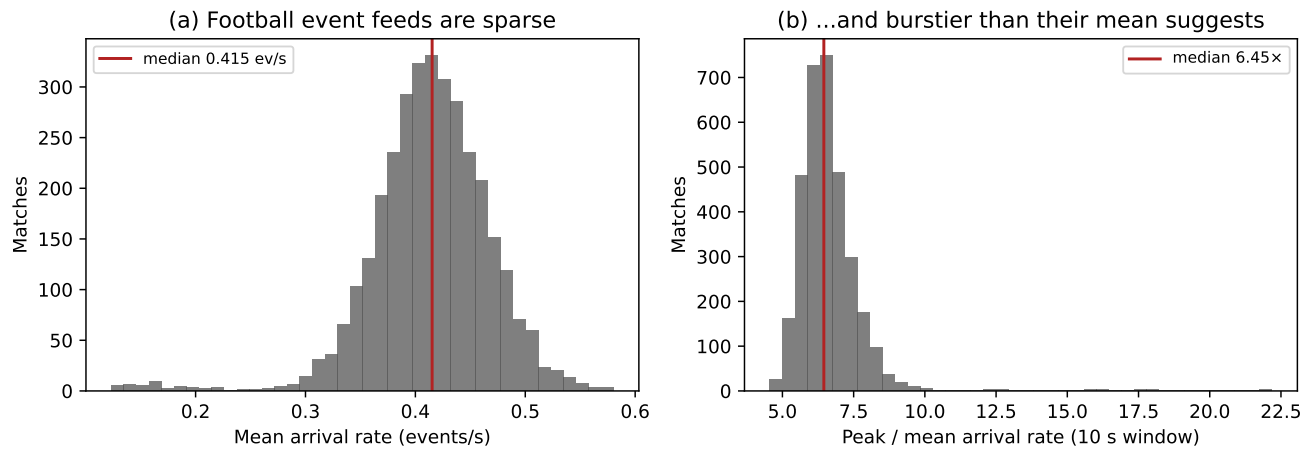


Figure 1. The football event feed as a workload, across 3,315 StatsBomb matches from 2003–2023. (a) Mean arrival rate per match. The distribution is tight around 0.415 events per second, a very low rate for a message broker. (b) The ratio of peak rate (over any ten-second window) to mean rate. Provisioning from the mean under-provisions by more than sixfold.

Table 2. Football event-feed arrival statistics, medians across matches. Peak rate is the maximum over any sliding ten-second window. The peak-to-mean ratio is the quantity that sizes a buffer.

Scope	Matches	Mean rate (ev/s)	Peak rate (ev/s)	Peak/mean
All competitions	3,315	0.415	2.70	6.45
La Liga	867	0.434	2.70	6.31
Ligue 1	435	0.430	2.70	6.32
Premier League	418	0.407	2.60	6.50
Serie A	380	0.423	2.70	6.35
1. Bundesliga	340	0.423	2.70	6.42
FA Women's Super League	326	0.397	2.60	6.55
FIFA World Cup	128	0.404	2.65	6.61
Women's World Cup	116	0.386	2.60	6.82
Indian Super League	115	0.346	2.60	7.64
African Cup of Nations	52	0.342	2.55	7.89
Men's football	2,806	0.419	2.70	6.43
Women's football	509	0.392	2.60	6.61

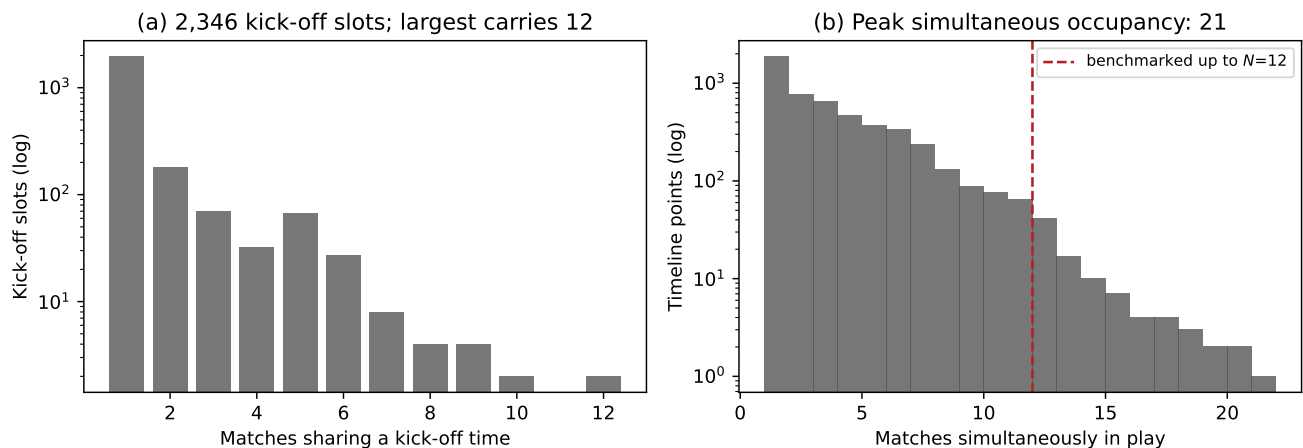


Figure 2. Concurrency derived from real kick-off schedules rather than chosen by hand. (a) Distribution of the number of matches sharing a kick-off instant, over 2,346 slots; the largest carries twelve. (b) Number of matches simultaneously in play, allowing a 115-minute match; the peak is 21. Both panels use a log count axis. Football never approaches the hundreds of concurrent feeds that general-purpose streaming benchmarks sweep.

In the concurrency experiment each of the N feeds carries a *distinct* real match at its true event rate. This matters. An earlier design gave every feed the same merged ten-match

plan, which meant that raising N raised the total event rate roughly tenfold per step: the concurrency axis was covertly a throughput axis.

Replay speed is a treatment in this study rather than a convenience. Headline results are at true real time, which is both the ecologically correct choice and, as Section 6 shows, the only regime in which our instrument is sound. Where we report accelerated results we say so and give their integrity status.

4.3 What we measure

The primary measure is **Time-to-Insight** (TTI): the interval from an event’s scheduled emission to its availability at the consumer. This is the end-to-end delay of the paper’s title. We decompose it into two parts that answer different questions (Figure 3):

$$\text{TTI} = \underbrace{(t_{\text{send}} - t_{\text{sched}})}_{\text{scheduling lag}} + \underbrace{(t_{\text{recv}} - t_{\text{ack}})}_{\text{broker transport}} + (\text{residual}).$$

Scheduling lag is the delay between when an event was due to be sent and when the producer actually issued the send. It measures the client and the load generator. *Broker transport* is the delay between the broker acknowledging a message and the consumer receiving it. It measures the broker. TTI is what a consumer experiences; transport is what distinguishes the two products. The distinction is not cosmetic, and it is the reason this paper reaches a different conclusion from its own first draft: for football feeds the two measures have opposite answers, and we report both.

The acknowledgement time t_{ack} is recorded by the KAFKA producer from its send callback and by the REDIS producer on return from XADD. It is therefore stamped in the *producer* process, while t_{recv} is stamped in the *consumer* process. That is the subtraction that can go wrong.

4.4 The clock-integrity check

Because transport subtracts a timestamp taken in one process from one taken in another, it admits a check that no statistic can supply: the result cannot be negative. A message cannot arrive before it is sent Lampert (1978).

We implement this as a stated rule in `scripts/clock-integrity.py`, fixed before the final campaign and applied to every run in the study, including runs whose results looked entirely reasonable. A run is **rejected** if either:

1. more than 1% of its events carry a negative value in any latency component, or
2. the median of any component is negative.

A condition is reported as usable only if all of its runs survive. Where a condition is partly rejected we report the surviving runs and state the retention rate, and where retention is low enough to constitute selection we say so. The tool exits with a non-zero status when any run fails, so a campaign script can stop on it.

On the choice of threshold. The 1% figure is a judgement, and we report its consequences rather than defending it as obvious. Section 6.3 gives the full sensitivity analysis. In summary: the threshold materially changes how many individual runs are rejected, but it does not change which conditions are usable or which conclusions survive, across the whole range from zero tolerance to 20%.

4.5 Statistical methods

Tests. Latency distributions are non-normal, so difference tests are rank-based: Mann–Whitney U Mann and Whitney (1947) per condition, Kruskal–Wallis Kruskal and Wallis (1952) across concurrency levels, with rank-biserial effect sizes.

Estimator. We report the Hodges–Lehmann shift Hodges and Lehmann (1963) with a percentile bootstrap interval Efron (1979) as the primary estimate of the difference between two systems, rather than a difference of means. This corrects our own earlier practice and is forced by the data: the KAFKA producer’s scheduling lag is heavily skewed (median 0.24 ms but 95th percentile 88 ms in the single-machine corpus), so a mean-based comparison contrasts two differently contaminated estimators. The Hodges–Lehmann shift is also the estimator consistent with the rank tests used elsewhere. Where the three procedures — Welch, bootstrap and Hodges–Lehmann — disagree, we report the disagreement rather than choosing one.

Equivalence. Several claims here are negative. Because failing to reject equality is not evidence of equality, we state those claims using two one-sided tests Schuirmann (1987); Lakens (2017): the 90% confidence interval of the difference must lie entirely inside $(-\delta, +\delta)$ for the systems to be declared equivalent within δ .

Margins. Margins are fixed before testing and chosen in proportion to the quantity compared, which is a change from our earlier practice of applying one margin to every measure. For *broker transport*, a quantity of order one millisecond, $\delta = 1$ ms. For *end-to-end* TTI, $\delta = 40$ ms, one broadcast video frame at 25 frames per second, on the grounds that a difference smaller than one frame cannot change what a coach, broadcaster or model does. We report the margin, the observed difference and the interval in every case, and we note explicitly where a stricter margin would not be met.

Multiple comparisons. Holm’s procedure Holm (1979) controls the family-wise error rate at $\alpha = 0.05$ within each family. Families are declared over the design actually executed: the four per- N comparisons on transport form one family; the two Kruskal–Wallis tests are single omnibus tests, one per system, and are not corrected; the network-delay arm forms a separate family. Equivalence tests are pre-specified claims with their own margins and are not members of any family.

Power. Surviving samples range from 8 to 35 runs per cell. At eight runs per system — the $N=1$ cell, which the integrity check reduced most — power is below 0.4 even for large effects Cohen (1988), and we treat that cell as descriptive and say so wherever it appears. Cells with 19 or more runs reach power above 0.75 for large effects.

4.6 A map of the study

Because a substantial part of this study is reported as invalid, Table 3 states up front which experiment supports which claim and which experiments were rejected.

5 The Answer We Obtained First

This section reports the study as it stood before the integrity check existed. We present it in full rather than describing it,

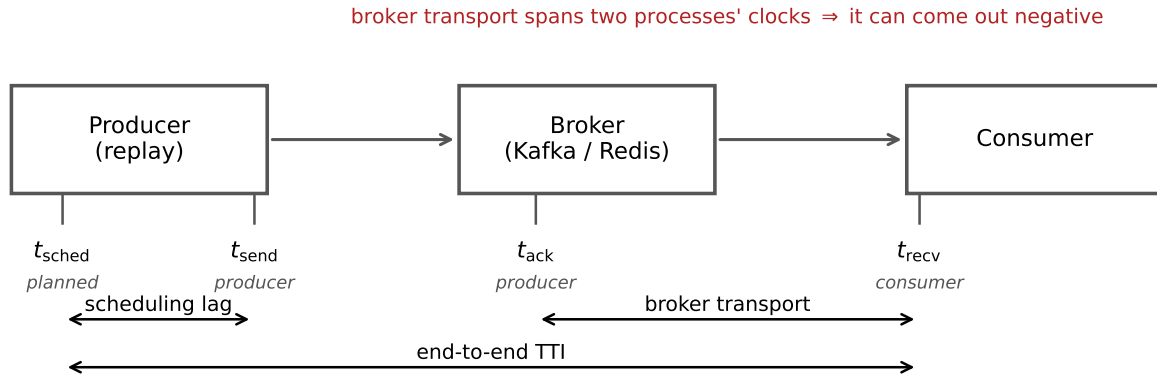


Figure 3. The replay pipeline, the four timestamps we record, and the intervals each measure spans. Scheduling lag is measured entirely inside the producer process. Broker transport subtracts a time recorded by the producer from a time recorded by the consumer, which is why it can come out negative when either process is delayed — the failure this paper is about.

Table 3. Map of the study. Every arm is listed, including those the integrity check rejected, so that the evidence behind each claim can be traced. “Retained” is runs surviving the check.

Arm	Testbed	Replay	Retained	Supports
Workload characterisation	—	—	n/a	Sections 3.1, 3.2
Concurrency $N \in \{1, 9, 10, 12\}$	B	real time	164/201	RQ1, RQ2, RQ3 (Section 7.1)
Network delay 5/20/50 ms	B	10×	REDIS 45/45; KAFKA 41/45	RQ4 (Section 7.3)
Acknowledgement batching, $N=1$	B	real time	8/8	RQ4 (Section 7.3)
Acknowledgement batching, $N=5$	B	10×	REDIS 60/60	unexplained null (Section 7.3)
Connections $N=10$	B	real time	KAFKA 10/10; REDIS 7/10	Section 7.4
<i>Rejected by the integrity check; reported but supporting no claim</i>				
Concurrency $N \in \{1, 5, 10\}$	A	10×	0 conditions	RQ5 (Section 5)
Throughput sweep	A	varied	0 conditions	—
Synthetic network delay	A	10×	0 conditions	—
Concurrency $N \in \{1, 5, 10\}$	B	10×	0–2 of 15–30 runs	—
Connections $N \in \{25, 50, 100\}$	B	real time	8–24 of 25–100 runs	—
Three-node cluster	B	10×	0/15 both systems	—
Durability (acks, appendfsync)	A	10×	0 conditions	direction only

for two reasons. It is the evidence for RQ5. And a reader cannot judge how convincing an invalid result set can be from a summary of one.

5.1 The result

On Testbed A, replaying distinct real matches at ten times real speed across one, five and ten concurrent feeds with 15 to 30 runs per cell and both producers pipelined, broker transport separated the two systems cleanly (Table 4). REDIS rose steadily from 1.006 to 1.346 ms, an increase of 34% across the range. KAFKA was flat to within 0.3%. Every comparison was significant after Holm correction, the strongest at $p = 9.0 \times 10^{-11}$, and at five feeds and above the two systems’ run distributions did not overlap at all.

The finding was not merely significant, it was explicable, for the architectural reason set out in Section 2.1. The measurement appeared to reproduce the textbook distinction between a single-threaded server and a partitioned log, at the scale one would predict and in the direction one would predict.

5.2 The checks it had already passed

The corpus behind Table 4 was not naive. It was the product of five earlier corrections, each of which had reversed the apparent winner, and each of which we found by the ordinary means of investigating a result that looked wrong.

1. *A process-relative clock.* Stamping cross-process events with Python’s `perf_counter_ns`, which measures from an arbitrary origin per process, injected each run’s consumer start-up offset (about two seconds) into every measurement. Fixed by using `time.time_ns`, a shared wall-clock epoch; transport then fell to about one millisecond.
2. *A saturating replay speed.* At 120 times real speed the producer could not keep pace, inflating scheduling lag to tens of seconds. Reduced to a nominally non-saturating ten times.
3. *An asymmetric load generator.* The KAFKA producer ran synchronously, blocking on a broker round trip for every event, while the REDIS producer dispatched asynchronously through a worker pool. At one feed, median TTI fell from 1,644 ms to 16 ms once the

Table 4. The first result set, later rejected. Broker transport (median, ms) by concurrency on Testbed A at 10× replay. Every cell fails the clock-integrity check of Section 4.4. The table is retained because this paper’s argument concerns how convincing it looks, not its content. Section 7.1 supersedes it.

N	KAFKA	REDIS	Difference	Mann–Whitney p_{Holm}	Integrity
1	0.999	1.006	0.007	1.0×10^{-4}	rejected
5	1.001	1.197	0.196	6.5×10^{-6}	rejected
10	1.002	1.346	0.344	9.0×10^{-11}	rejected

KAFKA producer was pipelined. Both producers are now configured comparably.

4. *A concurrency axis that was covertly a throughput axis.* Giving every feed the same merged ten-match plan raised the aggregate event rate roughly tenfold per concurrency step. Fixed by giving each feed a distinct match at its true rate.
5. *A heavy tail contaminating every mean.* KAFKA’s end-to-end 95th percentile sat at 99–105 ms at every concurrency level while its median moved only from 1.8 to 7.2 ms. A 95th percentile that ignores load is not system behaviour. Decomposition located it in the producer’s scheduling lag rather than in transport. We addressed it by moving to the Hodges–Lehmann estimator rather than by excluding a warm-up period, which we tested and rejected because only about a third of the slow events fall in the first 5% of a run.

We also verified that the comparison was not biased by message size or protocol: payloads are near-identical across systems (median about 170 bytes) and serialisation costs are negligible and comparable (about 2.3 and 2.1 microseconds for KAFKA, 2.4 and 2.2 for REDIS).

A sixth problem belongs here because it shaped our later protocol. Our first run of the acknowledgement-batching experiment of Section 7.3 reported no improvement at all — a clean, publishable-looking refutation of our own hypothesis. In fact the treatment had never reached the consumer: the orchestration script accepted the option but, because of a failed edit, never passed it on. We detected this only by deliberately supplying an invalid option and observing that the consumer did not reject it. A null result produced by a treatment that was never applied is indistinguishable, in the data, from a genuine one. Every experiment reported here now carries a manipulation check that stops the campaign if the treatment is not accepted, and the consumer logs its effective configuration for each run.

The point of this list is not the corrections themselves. It is that a corpus which had survived six of them, a fairness audit, and a full battery of rank tests, effect sizes and multiplicity correction, was still entirely invalid — and nothing in its output said so.

6 The Integrity Audit

6.1 What the check rejects

Applying the rule of Section 4.4 to all 2,266 runs gives Table 5. On Testbed A, 862 of 1,382 runs are rejected and only 8 of 76 conditions survive intact. On Testbed B, 459 of 884 runs are rejected and 13 of 40 conditions survive. No condition behind Table 4 survives.

6.2 Why it fails, and why it looked fine

The cause is visible in *which* conditions fail. Every rejected condition in the concurrency corpus was replayed at ten times real speed; every condition replayed at true real time passes.

The mechanism follows from Section 2.3. The acknowledgement time is recorded by the producer’s callback thread. When accelerated replay saturates the driver machine’s CPU, that thread is descheduled, and by the time it runs and reads the clock the consumer has already recorded receiving the message. The result is a negative transport value. It is a timestamping race under load, not a broken or unsynchronised clock, which is why medians stay positive at moderate load and invert wholesale only when the machine saturates. At one hundred connections in our connection sweep the failure becomes gross enough to see without any check at all: KAFKA’s median transport is reported as -6.4 ms.

The reason the corpus looked healthy is arithmetic. In the conditions behind Table 4, between 5 and 14% of individual events invert. That is enough to displace a median by a few tenths of a millisecond — in a measurement whose entire effect is 0.34 ms — and far too little to make any aggregate look wrong. Every median stays positive. Every confidence interval stays narrow. The bias is also *asymmetric* between the two systems, because their clients record the acknowledgement differently: KAFKA from an asynchronous callback, REDIS on return from a blocking command. An asymmetric bias between two arms of a comparison manufactures a difference where none exists. The result was a large, significant, theory-confirming effect assembled almost entirely out of instrument failure.

6.3 How much the threshold matters

We had initially expected the distribution of inversion rates to be clearly bimodal, with sound runs at zero and broken runs at obviously high rates, which would have made the 1% threshold an uncontroversial choice. It is not (Figure 4a). Only 104 of 1,382 runs are completely clean, and the remainder spread over three orders of magnitude with no natural gap. The threshold is therefore a real decision, and we report its consequences (Table 6, Figure 4b).

Two things follow, and they point in different directions. The threshold strongly affects *how many runs* are rejected: from 92.5% at zero tolerance to 19.2% at a very permissive 20%. But it does not affect *which conditions are usable*, and therefore does not affect any conclusion in this paper. A condition is usable only if all of its runs survive, and even at a 5% threshold the accelerated concurrency arms retain only 32 to 77% of their runs, so no such condition qualifies at any threshold we considered. The withdrawal of Table 4 therefore holds across the entire range.

Table 5. Clock-integrity audit of every run in the study. A run is rejected if more than 1% of its events carry a negative value in any latency component, or if any component median is negative. A condition is usable only if all of its runs survive.

Corpus	Runs	Rejected	%	Conditions	Usable
Testbed A (single machine)	1,382	862	62.4%	76	8
Testbed B (four machines)	884	459	51.9%	40	13
Total	2,266	1,321	58.3%	116	21

We prefer to report this than to claim a robustness the data does not show. A reader who thinks 1% too strict or too lenient can read the consequence directly from Table 6.

Table 6. Sensitivity of the audit to the rejection threshold, on the 1,382 runs of Testbed A. The threshold changes how many runs are rejected but not which conditions are usable, because a condition requires all of its runs to survive.

Threshold	Runs rejected	% of runs
0% (zero tolerance)	1,278	92.5
0.1%	1,086	78.6
0.2%	1,037	75.0
0.5%	937	67.8
1% (used here)	862	62.4
2%	763	55.2
5%	594	43.0
10%	445	32.2
20%	265	19.2

6.4 The property that makes this general

One feature of the audit deserves emphasis because it transfers beyond this study. The check is a test against zero, so it rejects a measurement in proportion to how close that measurement sits to zero. Our network-delay arm, where REDIS transport is tens of *seconds*, passes the check almost perfectly — 15 runs out of 15 at every injected delay — on the same hardware, in the same campaign, minutes apart from conditions that fail outright. A few milliseconds of timestamping race is invisible against a 31-second effect and fatal against a 0.34-millisecond one.

The check therefore bites hardest exactly where the scientific question is most delicate. This inverts a common intuition: that a large, clean, highly significant effect is the trustworthy kind. In cross-process delay measurement, an effect of the same order as the instrument’s own noise floor is the one most likely to have been manufactured by it, and statistical significance offers no protection, because the artefact is systematic rather than random.

We recommend that streaming benchmarks report an integrity audit alongside their results as a matter of course, in the way a survey reports its response rate.

6.5 What we therefore withdraw

We withdraw the entire Testbed A arm: the concurrency comparison, the throughput sweep, the synthetic network-delay arm, the acknowledgement experiment conducted there, and the decision-staleness aggregates computed from them. On Testbed B we withdraw the accelerated concurrency sweep, the connection sweep above ten

connections, and the three-node cluster arm, which fails on every run in both systems and therefore supports nothing.

We report this at length rather than quietly rebuilding the paper around the surviving data, because the withdrawn result is this study’s most transferable finding. It was more publishable than the result that replaced it.

7 Results

Everything in this section is measured on Testbed B and passes the check of Section 4.4. Each table states its retention.

7.1 End-to-end delay at real football concurrency (RQ1, RQ2)

Concurrency levels are those derived in Section 3.2, $N \in \{1, 9, 10, 12\}$, each feed carrying a distinct real match at true real time. Of 201 runs measured, 164 survive the integrity check. KAFKA passes in full at every level (8, 27, 30 and 35 runs); REDIS passes in full at one feed and is screened run by run above it (20 of 27, 17 of 30, 19 of 36), which we flag as partial selection and account for below.

REDIS *delivers a twentyfold lower end-to-end delay*. Median TTI is 105.5 ms for KAFKA and 5.2 ms for REDIS, pooled across levels, and the separation is complete at every concurrency level tested (Table 7, Figure 5a). No KAFKA run falls below 104 ms and no REDIS run exceeds 11 ms. Both systems clear every budget in Table 1, but with very different headroom: KAFKA meets the tightest of them, in-play betting at 200 ms, by a factor of two, while REDIS meets it by a factor of forty.

Neither system slows down as matches are added. Broker transport is flat across the entire realistic range. Kruskal–Wallis gives $p = 0.061$ for KAFKA and $p = 0.091$ for REDIS, neither significant, and the ratio of largest to smallest median is 1.06 and 1.17 respectively. This directly contradicts Table 4, and the contradiction is the audit’s doing rather than a difference of interpretation.

The two brokers are equivalent to within one millisecond. At $N \in \{9, 10, 12\}$ all three procedures agree that transport is equivalent within the 1 ms margin, with Hodges–Lehmann shifts of 0.021, 0.116 and 0.053 ms. At $N=1$ the procedures disagree: Hodges–Lehmann establishes equivalence with a shift of 0.081 ms, while the mean-based and bootstrap procedures do not, because a single KAFKA run carries a 6.3 ms start-up outlier that pulls the mean to 1.49 ms. With eight runs per system we report the disagreement rather than resolving it. $N=1$ is also the only level at which the difference test is significant ($p_{\text{Holm}} = 0.019$, rank-biserial -0.81), which given eight runs and a

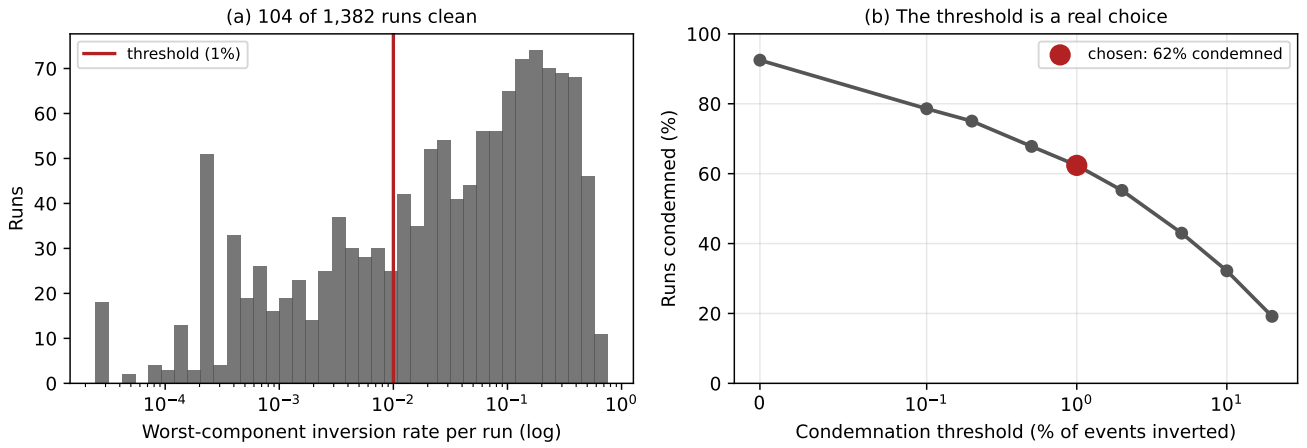


Figure 4. The integrity audit on Testbed A's 1,382 runs. (a) Distribution of the worst component's inversion rate per run. Only 104 runs are completely clean and the rest spread over three orders of magnitude, so there is no natural threshold. (b) Share of runs rejected as a function of the threshold. The choice matters for the run count and not for any conclusion, since a usable condition requires every one of its runs to survive.

0.07 ms median difference we read as a small-sample artefact rather than a finding.

7.2 Where the difference actually comes from (RQ3)

TTI and transport give opposite answers, and the decomposition explains why (Figure 5b). Of KAFKA's 105.5 ms, 102.9 ms is **producer scheduling lag**, against REDIS's 1.8 ms. Broker transport is 0.84 ms and 0.80 ms respectively. The entire twentyfold difference occurs before either broker is involved.

Three properties of this offset are measured, and together they rule out the obvious explanations:

- It is *constant*. The 50th and 95th percentiles of TTI differ by under 0.05 ms within a run, so every event pays it. It is not a tail effect.
- It is *unaffected by concurrency*: 103.0, 103.0, 102.8 and 102.9 ms at one, nine, ten and twelve feeds. It is not contention.
- It *depends on the arrival rate*. Replaying the identical data, code and machines at ten times real speed reduces KAFKA's scheduling lag from 103 ms to between 0.19 and 1.01 ms, and its TTI to between 1.8 and 7.2 ms.

The explanation most consistent with all three is that the KAFKA client's background sender thread must wake from idle to dispatch a record and pays a fixed cost of order 100 ms when the producer queue has been empty — which, at football's 0.415 events per second, is the case before essentially every event. Under accelerated replay events arrive closely enough that the sender is never idle, and the cost disappears. We flag that this explanation is inferred from the three measured properties rather than from instrumentation of the client's internals. We state it as the leading hypothesis rather than an established mechanism; separating it from a metadata-refresh or retry-backoff timer would require client-level tracing we did not perform.

What does not depend on the mechanism is the consequence, and it is this paper's sharpest methodological

result after the integrity check itself. **The difference between these two systems on a football feed exists only at football's arrival rate.** A benchmark that drives them with a dense synthetic publisher — which is to say every benchmark in Section 2.2 — measures the regime in which the difference is absent, and reports parity. The sparse, bursty, mostly-idle pattern described in Section 3.1 is not a nicety of ecological validity; it is the condition under which the effect exists at all.

7.3 The condition that reverses the ordering (RQ4)

Every result so far places consumer and broker on separate machines but on a fast private network. We therefore injected one-way delay at both brokers using `tc netem`, at 5, 20 and 50 ms applied identically to each, and repeated the five-feed condition. The REDIS arm passes the integrity check in full at every delay, for the reason given in Section 6.4: the effect is four orders of magnitude larger than the instrument's noise floor.

The ordering reverses completely (Table 8, Figure 6). KAFKA tracks the injected delay almost additively, its transport rising from 5.6 to 20.8 to 44.9 ms against injected delays of 5, 20 and 50 ms. REDIS collapses: 4.6 seconds at 5 ms of delay, 31.3 seconds at 20 ms and 102.5 seconds at 50 ms, roughly 900, 1,560 and 2,050 times the delay actually injected. No run was truncated, so this is amplification rather than loss. Against Table 1, KAFKA at 20 ms of delay still meets every budget listed; REDIS misses all of them by four orders of magnitude.

The explanation, and an experiment that tests it. Numbers of this size cannot be a per-event cost: 102 seconds cannot be assembled from 50 ms applied once per event. The account we test is a *round-trip-bound consumption pattern*. Our REDIS consumer acknowledges each entry with `XACK` and waits for the reply before continuing — the idiomatic pattern in client tutorials — so a read returning two hundred entries is followed by two hundred sequential round trips. On a fast network this is free: a round trip on an established connection costs 0.22 ms, a ceiling of about 4,200 exchanges

Table 7. End-to-end delay and its decomposition at the concurrency levels football produces. Testbed B, true real-time replay, after the integrity check (164 of 201 runs retained). Equivalence is tested on transport against a 1 ms margin proportionate to the measurement.

N	TTI median (ms)		Scheduling lag (ms)		Transport (ms)		Transport equivalent within 1 ms
	KAFKA	REDIS	KAFKA	REDIS	KAFKA	REDIS	
1	105.3	5.0	103.0	1.4	0.792	0.721	procedures disagree (8 runs/system)
9	105.5	5.4	103.0	1.9	0.802	0.807	yes (all three)
10	105.3	5.8	102.8	2.0	1.000	0.855	yes (all three)
12	105.7	5.3	102.9	1.7	0.839	0.844	yes (all three)

Pooled: KAFKA TTI 105.5 ms (100 runs), REDIS 5.2 ms (64 runs). Across N: KAFKA $p = 0.061$, REDIS $p = 0.091$; neither significant.

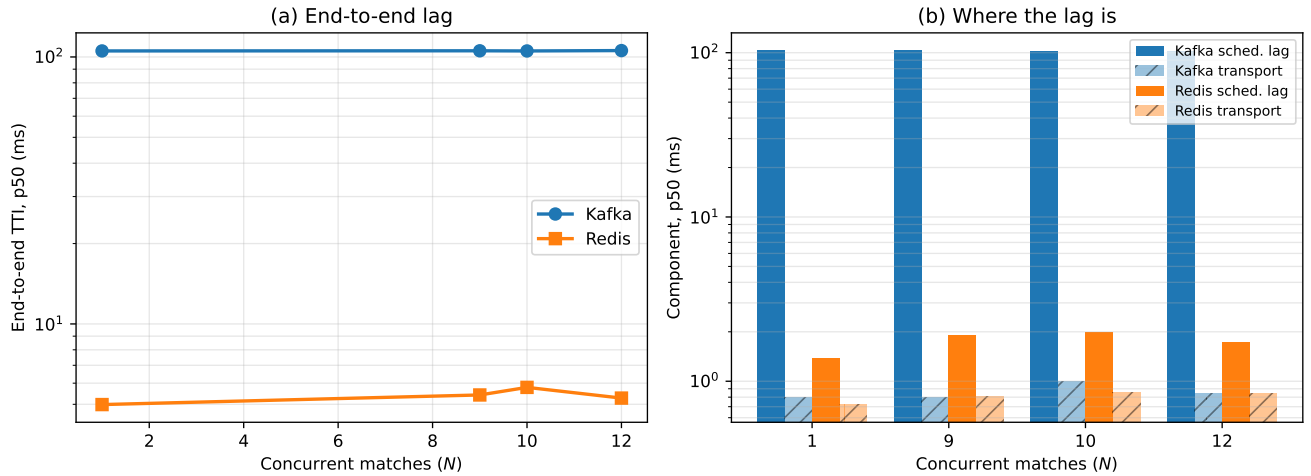


Figure 5. End-to-end delay at real football concurrency, Testbed B, after the integrity check. (a) REDIS delivers a median around 5 ms against KAFKA’s 105 ms, flat across every concurrency level football produces. (b) The same runs decomposed. The twentyfold difference lies entirely in producer scheduling lag; the two brokers’ own transport is separated by well under a millisecond. Log scale on both panels.

Table 8. Injected one-way broker delay, applied identically to both systems, five feeds, Testbed B. KAFKA tracks the delay; REDIS amplifies it by three orders of magnitude. Integrity retention is given per cell. The zero-delay row is rejected in both systems and is shown only to mark the absence of a usable baseline in this arm.

Injected delay	KAFKA		REDIS		Retained	
	TTI	Transport	TTI	Transport	KAFKA	REDIS
0 ms	<i>rejected</i>		<i>rejected</i>		0/15	0/15
5 ms	12.4 ms	5.6 ms	4,651 ms	4,645 ms	14/15	15/15
20 ms	77.8 ms	20.8 ms	31,401 ms	31,268 ms	12/15	15/15
50 ms	336.6 ms	44.9 ms	103,143 ms	102,460 ms	15/15	15/15

per second against a demand of well under one per second. At 20 ms it has no headroom at all. The queueing statement is elementary [Kleinrock \(1975\)](#): once the service rate falls below the arrival rate the queue grows without bound, and the delay is then set by the accumulated backlog rather than by the network. KAFKA’s consumer serves `poll()` from a prefetched buffer and commits its position on a timer, so it has no per-record round trip to expose.

This is testable by intervention rather than observation, and we stated the criterion in advance: if amortising the acknowledgements does not help, the account is wrong. At one feed under true real-time replay — the realistic football condition — batching acknowledgements in groups of two hundred takes median TTI from 4,138 ms to 103 ms, a factor of **40.2**, replicated four times with a spread of 4 ms in the unbatched arm and 0.4 ms in the batched one (Table 9).

Instrumenting the consumer’s read loop confirms the mechanism and corrects its shape. We had supposed the consumer was limited to about one message per round trip. It is not: under delay each `XREADGROUP` returns a *full* batch, a median of 106 messages, in about 32 ms. The constraint is entirely in the acknowledgement path. With per-message acknowledgement the consumer spends roughly $200 \times 20 \text{ ms} = 4 \text{ s}$ acknowledging each batch, issuing no reads at all meanwhile: across a whole match it completes 13 reads, of which 4 return data, each finding a large accumulated backlog. With acknowledgements amortised the same consumer performs 37 reads returning a median of 16 messages each, and keeps pace.

An open question we do not resolve. At five feeds under accelerated replay the same change does nothing: 68,960 $\rightarrow$ 73,598 ms at 20 ms of delay and 87,624 $\rightarrow$ 92,386 ms at

Table 9. The acknowledgement-batching experiment. At one feed and true real time — the football condition — batching recovers a factor of 40.2, and read-loop instrumentation shows why. At five feeds under accelerated replay the same change, with a passing manipulation check and full integrity retention, produces no improvement; we report this as unexplained. Read-loop columns were recorded only for the one-feed arm.

Condition	Arm	TTI median	Reads	Non-empty	Msgs/read	Improvement
1 feed, real time, 20 ms delay	per-message batched (200)	4,138 ms 103 ms	13 37	4 25	106 16	40.2×
5 feeds, 10×, 20 ms delay	per-message batched (200)	68,960 ms 73,598 ms	— —	— —	— —	none (0.94×
5 feeds, 10×, 50 ms delay	per-message batched (200)	87,624 ms 92,386 ms	— —	— —	— —	none (0.95×

One-feed replicates: unbatched 4137.9/4133.9/4138.2/4138.0 ms; batched 103.0/103.3/103.0/103.1 ms.

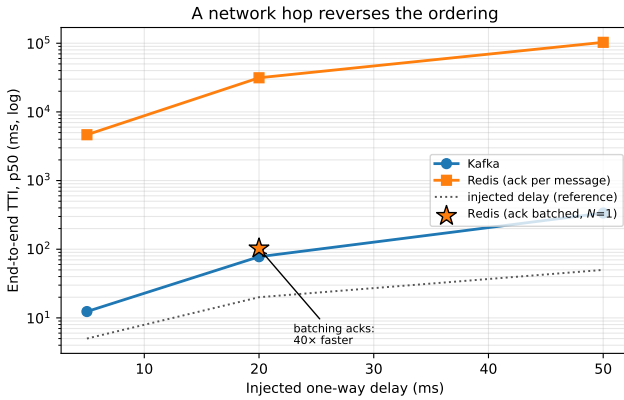


Figure 6. The condition that reverses the ordering. KAFKA tracks the injected delay; REDIS with per-message acknowledgement amplifies it by three orders of magnitude. The star marks the same REDIS consumer with acknowledgements batched two hundred at a time, at one feed and true real time: the collapse disappears. Log vertical scale.

50 ms, both marginally *worse* when batched. We report this prominently because the obvious explanations do not survive inspection. The treatment was applied: the campaign script runs a manipulation check that stops if the option is not accepted, and the consumer logs its effective batch size for every run. The measurement is sound: the REDIS arm passes the integrity check on all 15 runs in each of the four cells. The machine is not saturated in general: KAFKA in the very same runs sits at 78 ms. An earlier version of this paper explained the null away as an integrity failure. That explanation is false and we withdraw it.

The account we find most plausible is that at five feeds and ten times real speed the consumer is far outside the stable regime in both arms, so the binding constraint has moved somewhere the acknowledgement path no longer dominates. But we did not instrument the read loop in that arm and cannot demonstrate it. We therefore claim the round-trip-bound mechanism *at realistic football load*, where we have both the intervention and the read-loop evidence, and we state plainly that its behaviour at five times that load is unexplained. It is the most important open question this study leaves.

7.4 Connections and clustering

Holding the per-feed rate at real time and varying only the number of concurrent connections, transport stays near one millisecond for both systems at ten connections (KAFKA 1.01 ms, 10 of 10 runs retained; REDIS 0.84 ms, 7 of 10), and end-to-end delay preserves the ordering of Section 7.1 (105.4 against 5.3 ms). Above ten connections the check rejects both systems, and at one hundred the failure is gross: KAFKA’s median transport is reported as -6.4 ms. We therefore report no result above ten connections for this arm.

This is a real limitation, because it is the arm that would speak to a multi-league feed platform (Section 3.2). It is also the clearest instance in this study of the check removing an answer we wanted rather than one we disliked.

The three-node cluster arm fails on every run in both systems and supports no claim. We report its existence and nothing from it.

7.5 Durability

Stronger durability costs delay in both systems, as PACELC predicts Abadi et al. (2012): KAFKA’s `acks=all` raises median TTI relative to `acks=1`, and REDIS’s `appendfsync` always raises it substantially relative to `everysec`, reflecting a disk synchronisation on every write. The quantitative comparison was measured only on Testbed A and is therefore withdrawn. We report the direction, which the surviving data does not contradict, and note that a durability replication on Testbed B is the most obvious extension of this work.

7.6 Putting the numbers on a decision scale

A benchmark measures transport. A decision-maker experiences everything between the action on the pitch and the model’s ability to act on it:

$$\text{staleness} = \underbrace{\text{annotation}}_{\text{human coding}} + \underbrace{\text{delivery}}_{\text{measured here}} + \underbrace{\text{inference}}_{\text{model}}.$$

As set out in Section 2.4, no peer-reviewed measurement of annotation delay exists, so we sweep it from 1 to 30 seconds rather than adopting a vendor point estimate, and we set inference to zero, which flatters the delivery term rather than the reverse.

At a vendor-typical 15 seconds of annotation, KAFKA’s measured 105.5 ms of delivery is 0.70% of end-to-end

staleness and REDIS's 5.2 ms is 0.03%; the choice between them is worth 100.2 ms, or **0.66%**. Inverting the relation is more informative: for delivery to own even one percent of what a decision-maker experiences, annotation would have to complete within 10.4 s, and for ten percent, within 0.95 s. The first is inside the range vendors already claim; the second is what automated event detection from tracking data would have to achieve.

The comparison looks different again if one asks about the brokers rather than the whole pipeline. Broker transport of 0.84 ms is 0.006% of the same budget, and the difference between the two brokers is 0.04 ms, or 0.0003% — four orders of magnitude below visibility. This is the honest form of the claim that infrastructure does not matter for football: *broker* choice is structurally irrelevant, *client* configuration is worth two thirds of a percent, and a misconfigured consumer behind a network hop is worth more than the entire annotation budget.

Expressed on a decision scale, an event delivered with delay L leaves the consumer's forecast wrong by the probability mass that event moved, for L seconds Kaul et al. (2012). The largest forecast move in our corpus is a 95th-minute equaliser that shifts the win/draw/loss distribution by 0.955 in total variation. At REDIS's 5.2 ms that event costs 0.005 probability-seconds; at KAFKA's 105.5 ms, 0.101; and at the 31.4 s a round-trip-bound consumer suffers behind a 20 ms hop, about 30 probability-seconds. The first two are decision-irrelevant and the third is not.

We note two limits on this conversion. It changes units rather than adding inferential power: both systems are scored against the identical model, so the ordering under it is the ordering under delay. And the win-probability model behind it is a calibrated stand-in for the unreleased state of the art Robberechts et al. (2021) — its expected calibration error is 0.054 over 28,240 game states, but its skill over a lookup table conditioned on nothing but the current goal difference is only +0.026. It should be read as a calibrated yardstick, not as a model of in-play win probability in any stronger sense.

8 Discussion

8.1 Which system, under which conditions

The question the project set out to answer has a conditional answer, and the conditions can be known in advance (Table 10).

For a deployment on one network at football's event rates — the common case — REDIS Streams is the better choice on delay, by a factor of twenty end to end. The caveat is that this is a property of the client rather than the broker, so a KAFKA deployment that tunes its producer out of the idle-wakeup regime should close most of the gap. We did not achieve that, and a reader who does would find the two systems equivalent, since the brokers themselves are equivalent to within one millisecond.

For any deployment where a consumer sits across a meaningful network, KAFKA is the safer choice — not because its broker is faster, but because its client already amortises what REDIS's idiomatic consumer does per message. A REDIS deployment can match it, and the fix is

one line, but the failure if you do not apply it is severe and invisible on a local testbed.

Neither system is under strain from football. Concurrency up to the twelve simultaneous matches the sport produces changes nothing for either, and both meet every published latency budget in every configuration we could measure soundly.

8.2 How hard each system is to configure

A latency comparison that ignores configuration effort misstates the engineering choice, so we report what it actually took. The finding is symmetric, and we think it is the most practically useful result in the paper after the integrity check.

Each system has exactly one client setting that moves delay by one to two orders of magnitude, and in each case the introductory example in the documentation uses the slow value. For KAFKA it is producer pipelining: with one outstanding request per connection, which is what a synchronous send example gives you, median TTI at one feed was 1,644 ms; with `max.in.flight` set to 64 it was 16 ms, a factor of 103. For REDIS it is acknowledgement batching: the per-message XACK that every tutorial shows costs 4,138 ms behind a 20 ms hop, and batching two hundred at a time costs 103 ms, a factor of 40.2.

Both share a property that makes them hard to catch: they are *free on a local testbed*. Pipelining does not matter when the broker round trip is 0.2 ms, and neither does acknowledgement batching. A benchmark conducted on loopback prices both at zero and will therefore certify as equivalent two clients that differ by two orders of magnitude in deployment. This is the practical corollary of Section 6.4: the conditions that make a testbed convenient are the conditions that make it least informative.

Beyond these, the two systems differ in the *shape* of their configuration surface rather than its size. KAFKA's delay-relevant settings (`acks`, `linger.ms`, `max.in.flight`, `batch.size`, and the consumer poll timeout) are numerous, documented together, and mostly safe to leave alone once pipelining is enabled. REDIS's are fewer (`COUNT`, `BLOCK`, `appendfsync`, and acknowledgement granularity) but the consequential one is not a setting at all. It is a property of how the application's read loop is written, which no configuration audit will surface.

We also note one operational asymmetry we hit directly. Our own REDIS cluster client had loopback addresses hard-coded for cluster node discovery, which made a genuinely distributed cluster impossible to test until we fixed the client. That is a defect in our code rather than in REDIS, but it is exactly the kind of defect a co-located testbed never reveals.

8.3 Methodological implications

Three recommendations follow, in decreasing order of confidence.

First, **report a physical-consistency audit**. Any delay computed as a difference of timestamps taken in different processes admits a sign check, and that check costs nothing. We rejected 58% of our own runs with it, and every rejected run had passed every conventional check we knew to apply. We would encourage reviewers to ask for the audit, and

Table 10. Which system, under which condition, on the evidence of this study. “Configuration effort” is the number of non-default client settings needed to reach the stated delay.

Condition	Better choice	Margin	Configuration effort
Single feed, one network	REDIS	5 vs 105 ms end to end	REDIS: none; KAFKA: 1 setting, unresolved
Up to 12 feeds, one network	REDIS	unchanged across N	as above
Broker capability alone	neither	equivalent within 1 ms	—
Consumer across a network	KAFKA	78 ms vs 31 s at 20 ms	REDIS: 1 setting, 40×
Durability required	not tested on the surviving testbed		both expose one setting
More than 12 feeds	not established (arm rejected)		—

authors to report the retention rate the way a survey reports its response rate.

Second, **drive the benchmark at the domain’s arrival rate, not a convenient one.** Our central explanatory result exists only at football’s 0.415 events per second and vanishes at ten times that. This is the opposite of the usual concern. An unrealistically *slow* workload is normally thought to risk failing to stress the system; here the realistic rate exposed a difference that the accelerated rate concealed. Sparse, bursty, mostly-idle workloads are their own regime.

Third, **measure at a realistic round-trip time, and treat interventions as interventions.** The two configuration defects of Section 8.2 are both invisible on loopback, and the one experiment we ran without a manipulation check produced a clean false null. Both problems are cheap to prevent and expensive to discover.

8.4 Limitations

The surviving evidence is narrower than the design. The integrity check removed the throughput sweep, the connection sweep above ten connections, the cluster arm, the durability quantification and the whole of Testbed A. Two consequences are substantive. We cannot characterise behaviour at multi-league concurrency, which is the regime a commercial feed platform occupies. And our durability comparison is qualitative.

The 103 ms KAFKA offset is attributed, not proven. We measure that it is constant, unaffected by concurrency and dependent on arrival rate, which rules out contention and tail effects and is consistent with an idle-sender wakeup, but we did not instrument the client internals. If it is a property of the Python KAFKA client rather than of KAFKA itself, then our end-to-end result is a statement about that client, and we say so in Section 7.2. It reproduced across both testbeds, four concurrency levels and 164 runs, so it is not an accident of one machine; it may still be an accident of one library.

The five-feed acknowledgement null is unexplained, as set out in Section 7.3. We prefer to leave it visible rather than resolve it by assumption.

The concurrency ceiling is a lower bound. StatsBomb open data samples matches rather than enumerating league-seasons, and the structural bound is per league. Our result to twelve feeds describes a consumer of a single competition.

The decision-scale conversion is a lens, not a second measurement. It changes units, adds no inferential power, and the win-probability model behind it is calibrated but modest.

Replay is not live capture. Feeds replay recorded matches, and the upstream annotation pipeline is not modelled —

deliberately, since it is the dominant term and no peer-reviewed measurement of it exists. Our figures bound the delivery contribution to staleness, not end-to-end feed freshness.

9 Conclusion

We set out to benchmark REDIS Streams against Apache KAFKA for real-time football feeds, on open data, at concurrency levels derived from the sport rather than chosen by hand. We obtained a complete answer, and it was physically impossible.

The audit that discovered this is the paper’s principal contribution. A negative broker transport — a message received before it was sent — is proof of instrument failure, and checking for it rejected 1,321 of our 2,266 runs, including every condition behind a large, significant, theory-confirming result about REDIS slowing down under concurrency. The rejected corpus had passed every conventional check: plausible medians, narrow intervals, large effect sizes, the architecturally expected direction, and six prior rounds of correction. It failed only against arithmetic. We report the check as a short, open, reusable script, and we note the property that makes it general: because it is a test against zero, it bites hardest exactly where the measured effect is smallest, which is to say on the delicate comparisons that benchmark papers exist to make.

Inside the check, the original question has an answer. At every concurrency level football produces, REDIS Streams delivers a median end-to-end delay of 5.2 ms against KAFKA’s 105.5 ms, and neither system slows down as matches are added. But the brokers themselves are statistically equivalent to within one millisecond. The twentyfold difference lies entirely in the client send path, and it exists only at football’s sparse arrival rate: replay the same data ten times faster and it disappears. The ordering reverses behind a network hop, where KAFKA tracks the injected delay while a round-trip-bound REDIS consumer turns 20 ms into 31 seconds — until its acknowledgements are batched, which recovers a factor of 40.2 at realistic load and, unexplainedly, nothing at five times that load.

The practical message is therefore about configuration rather than product. Each system has one client setting worth one to two orders of magnitude in delay; in each case the introductory example uses the slow value; and both are free on a local testbed, which is where such settings are usually evaluated. Choose REDIS Streams for a football feed on one network, choose KAFKA if a consumer sits across a network you do not control, and in either case measure at the domain’s

real arrival rate, at a realistic round-trip time, and with a check that your timestamps obey causality.

Data and Code Availability

All code, replay plans, aggregated results and the analysis pipeline are openly available at <https://github.com/Gustolandia/streaming-latency-sports>, archived at [DOI to be inserted on release]. The event data are the StatsBomb open dataset, used under its CC BY-NC 4.0 licence; raw event files are not redistributed but are re-fetchable from a pinned upstream commit (3bfbffef1de5750ebd47d770be0bb924a10cde54f) with `scripts/fetch_statsbomb_corpus.py`, so the 52 competition-season corpus is reconstructible exactly.

The workload characterisation of Section 3 is produced by `scripts/characterize_feed.py` and `scripts/kickoff_concurrency.py`; the budget of Section 7.6 by `scripts/staleness_budget.py`; the audit of Section 6 by `scripts/clock_integrity.py`; and every figure by `scripts/make_paper_figures.py` and `scripts/make_el_figure.py`. Cloud campaign scripts are in `cloud/campaigns/`, parameterised so that a fresh testbed needs only a hosts file. Every run records its git commit and per-file SHA-256 in a `meta.json`, and `reproducibility/MANIFEST.json` pins the checksums of all code and configuration used to produce the reported results. A regression test (`tests/unit/test_manuscript_consistency.py`) recomputes every headline figure in this paper from the committed result files and fails if the manuscript and the data disagree.

Acknowledgements

Using StatsBomb open dataset. Open-source Apache Kafka and Redis projects. MIT License.

References

- Abadi D et al. (2012) The PACELC theorem: Consistency in distributed systems. Extension of CAP theorem addressing latency-consistency trade-offs.
- Arasu A, Cherniack M, Galvez E, Maier D, Maskey AS, Ryvkina E, Stonebraker M and Tippetts R (2004) Linear road: A stream data management benchmark. In: *Proceedings of the 30th International Conference on Very Large Data Bases (VLDB)*. pp. 480–491.
- Brewer EA (2012) Cap twelve years later: How the “rules” have changed. *IEEE Computer* 45(2): 23–29.
- Cohen J (1988) *Statistical Power Analysis for the Behavioral Sciences*. 2nd edition. Lawrence Erlbaum Associates.
- Confluent I (2025) Apache kafka benchmarks: Performance at scale. *Confluent Blog* URL <https://www.confluent.io/blog/apache-kafka-benchmarks/>.
- Corbett JC, Dean J, Epstein M, Fikes A, Frost C, Furman JJ, Ghemawat S, Gubarev A, Heiser C, Hochschild P et al. (2013) Spanner: Google’s globally distributed database. *ACM Transactions on Computer Systems* 31(3): 1–22.
- Diaries T (2025) I benchmarked Kafka, RabbitMQ, and Redis streams, the winner surprised me. URL <https://medium.com/@ThreadSafeDiaries/i-benchmarked-kafka-rabbitmq-and-redis-streams-the-winner-surprised-me-cf3f484eb7b2>. Accessed: June 15, 2026.
- Efron B (1979) Bootstrap methods: another look at the jackknife. *The Annals of Statistics* 7(1): 1–26.
- Hesse G, Matthies C, Perscheid M, Uflacker M and Plattner H (2021) ESPBench: The enterprise stream processing benchmark. In: *Proceedings of the ACM/SPEC International Conference on Performance Engineering*. pp. 201–212.
- Hodges JL and Lehmann EL (1963) Estimates of location based on rank tests. *The Annals of Mathematical Statistics* 34(2): 598–611.
- Holm S (1979) A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* 6(2): 65–70.
- Jain R (1991) *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. Wiley.
- JusDB (2025) Redis streams vs Kafka: Event streaming architecture comparison 2025. URL <https://www.jusdb.com/blog/redis-streams-vs-kafka-event-streaming-comparison-2025>. Accessed: June 15, 2026.
- Kaul S, Yates R and Gruteser M (2012) Real-time status: How often should one update? In: *2012 Proceedings IEEE INFOCOM*. IEEE, pp. 2731–2735. DOI:10.1109/INFOCOM.2012.6195689.
- Kleinrock L (1975) *Queueing Systems: Volume I - Theory*. Wiley-Interscience.
- Kreps J, Narkhede N and Rao J (2011) Kafka: a distributed messaging system for log processing. *Proceedings of the 6th ACM Symposium on Networked Systems Design and Implementation*.
- Kruskal WH and Wallis WA (1952) Use of ranks in one-criterion variance analysis. *Journal of the American Statistical Association* 47(260): 583–621.
- Labs R (2025) Redis streams performance benchmarks. *Redis Blog* URL <https://redis.io/blog/redis-streams-performance/>.
- Lakens D (2017) Equivalence tests: a practical primer for *t* tests, correlations, and meta-analyses. *Social Psychological and Personality Science* 8(4): 355–362.
- Lamport L (1978) Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM* 21(7): 558–565.
- Liu H, Hopkins W, Gomez AM and Molinuevo JS (2013) Inter-operator reliability of live football match statistics from opta sportsdata. *International Journal of Performance Analysis in Sport* 13(3): 803–821.
- Mann HB and Whitney DR (1947) On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics* 18(1): 50–60.
- Mills DL (1991) Internet time synchronization: the network time protocol. *IEEE Transactions on Communications* 39(10): 1482–1493.
- Mohammad M (2025) Analysis of design patterns and benchmark practices in Apache Kafka event-streaming systems. *arXiv preprint arXiv:2512.16146*.
- OpenMessaging Project, Linux Foundation (2018) The OpenMessaging benchmark framework. Open standard, multi-platform

- messaging performance benchmark. Available at openmessaging.cloud/docs/benchmarks/.
- OptiView D (2025) What is low latency video streaming?: The complete guide. URL <https://optiview.dolby.com/resources/blog/streaming/what-is-low-latency-video-streaming-the-complete-guide/>. Accessed: June 15, 2026.
- Pappalardo L, Cintia P, Rossi A, Massucco E, Ferragina P, Pedreschi D and Giannotti F (2019) A public data set of spatio-temporal match events in soccer competitions. *Scientific Data* 6(1): 236.
- Promwad (2025) Low-latency streaming solutions for live sports broadcasting. URL <https://promwad.com/news/low-latency-streaming-solutions-live-sports-broadcasting>. Accessed: June 15, 2026.
- Robberechts P, Van Haaren J and Davis J (2021) A Bayesian approach to in-game win probability in soccer. In: *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. pp. 3512–3521. DOI:10.1145/3447548.3467194.
- Sanfilippo S (2017) Redis streams: a new data type for real-time streaming. *Redis Labs Blog* URL <https://redis.io/topics/streams-intro>.
- Schuurmann DJ (1987) A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics* 15(6): 657–680.
- Shukla A, Chaturvedi S and Simmhan Y (2017) RIoT Bench: An IoT benchmark for distributed stream processing systems. *Concurrency and Computation: Practice and Experience* 29(21): e4257.
- Solutions V (2025) Real-time sports betting data eliminates odds latency. URL <https://www.v2solutions.com/blogs/real-time-sports-betting-data-odds-latency/>. Accessed: June 15, 2026.
- StatsBomb (2023) Statsbomb open data. *GitHub Repository* URL <https://github.com/statsbomb/open-data>.
- Stonebraker M, Cetintemel U and Zdonik S (2005) The 8 requirements of real-time stream processing. *ACM SIGMOD Record* 34(4): 42–47.
- Tucker P, Tufte K, Papadimos V and Maier D (2008) NEXMark: A benchmark for queries over data streams. Technical report, OGI School of Science and Engineering, Oregon Health and Science University.
- Ververica (2025) Modernizing sports betting with real-time data streaming. URL <https://www.ververica.com/blog/modernizing-sports-betting-technology-to-empower-live-odds>. Accessed: June 15, 2026.
- Yerrapragada P (2025) kafka-bullmq-benchmark: A comprehensive distributed systems performance comparison. URL <https://github.com/praneethys/kafka-bullmq-benchmark>. Includes white paper with methodology; Accessed: June 15, 2026.